

Last-Mile Delivery Tracker

Comprehensive Product Specifications, Architecture & Operations Manual

Live Project Deployments

- Live Web App: <https://last-mile-delivery-tracker-fawn.vercel.app> (<https://last-mile-delivery-tracker-fawn.vercel.app>)
- Live API Host: <https://last-mile-delivery-tracker-ekb3.onrender.com> (<https://last-mile-delivery-tracker-ekb3.onrender.com>)
- GitHub Repository: <https://github.com/ananyasingh180306-ctrl/last-mile-delivery-tracker> (<https://github.com/ananyasingh180306-ctrl/last-mile-delivery-tracker>)

1. Executive Summary & Objective

Logistics and last-mile delivery services represent a complex domain containing multi-dimensional billing parameters, real-time tracking demands, and dynamic dispatch coordination.

The **Last-Mile Delivery Tracker** is a comprehensive, enterprise-ready delivery management platform. It enables customers to register, instantly compute delivery costs based on volumetric weights and active zone mappings, and track packages in real-time. Simultaneously, it equips administrators with tools to override delivery states, manually re-assign shipments, register new agent profiles, adjust regional boundaries, and customize rate cards.

2. Core Features & Additions

A. Advanced Billing & Pricing Engine

The system replaces hardcoded logic with a dynamic, parameter-driven pricing engine:

- **Volumetric Weight Calculation:** Parcels are assessed using their length, width, and height to calculate dimensional bulk
$$\text{Volumetric Weight (kg)} = \frac{\text{Length (cm)} \times \text{Width (cm)} \times \text{Height (cm)}}{5000}$$
 The engine bills on the higher value between the package's scale weight and its volumetric weight:
$$\text{Billable Weight} = \max(\text{Actual Weight}, \text{Volumetric Weight})$$
- **Connection Routing:** Pickup and drop pincodes are dynamically checked against the database to classify connections as local (**INTRA-ZONE**) or cross-town (**INTER-ZONE**).
- **Immutable Financial Snapshots:** Once an order is confirmed, the full calculation breakdown is snapshotted as an immutable JSON string inside the order record to preserve the audit trail if rates change later.

B. Dynamic Agent Dispatching (Auto-Assignment)

- Uses a capacity-weighted geodesic distance calculation:
$$\text{Score} = \text{Distance} \times \left(1 + \frac{\text{activeCount}}{\text{maxConcurrent}}\right)$$
 This distributes delivery tasks across closer agents while protecting busy agents from workload fatigue.
- **Row-Level Transaction Locking:** Uses Prisma transaction queries with `FOR UPDATE` equivalent row locking during assignment to prevent race conditions from double-booking agents.

C. Real-Time Synchronization & Timeline Auditing

- **Socket.io WebSockets:** Pushes instant state changes to the customer's timeline animations without polling.
- **Immutable Status Logs:** Every lifecycle change records the actor, timestamp, and optional remarks in an append-only `OrderStatusHistory` table.

D. Failed Delivery & Rescheduling Flow

- If an agent marks a shipment as `FAILED` (e.g. *Recipient unavailable*), the customer can select a new delivery window.
- Rescheduling automatically releases the old agent (decrementing their active workload) and triggers the auto-dispatch engine to find the nearest available agent for attempt number two.

E. SMTP Email Notification Integration

- Uses Nodemailer to route transactional updates to customers and agents (configured for Mailtrap sandbox for safe local testing).

F. Admin Control Room & Configuration Panels

- **Shipment Overrides:** Bypasses typical lifecycle transitions to resolve exceptions.
- **Agent Registry:** Register new agents, allocate concurrent workloads, select coverage zones, and reset passwords.
- **Zone & Rate Configuration:** Create rate cards and set COD surcharges dynamically without code modifications.

3. Technology Stack

- **Frontend:** React (Vite SPA) + TypeScript + Tailwind CSS + Socket.io-client + Heroicons
- **Backend:** Node.js + Express + TypeScript + Prisma ORM + Socket.io + Nodemailer + Swagger API Docs
- **Database:** SQLite (local development) / PostgreSQL (production compatibility)
- **Containers:** Docker & Docker Compose support
- **Hosting:** Vercel (Frontend) + Render (Backend & DB)

4. Database Schema Structure

- **User:** Authentication credentials and role configuration (`ADMIN`, `CUSTOMER`, `AGENT`).
- **Agent:** Coordinates, coverage zones list, and concurrent capacity limits (`maxConcurrent`, `activeCount`).
- **Zone:** Service areas with logical name groupings.
- **AreaZoneMapping:** Maps postal pincodes and neighborhoods to their logical logistics zones.
- **Order:** Dimensions, connection types, immutable cost snapshots, live statuses, and current agent.
- **OrderStatusHistory:** Immutable audit logs capturing timestamps and actors.

5. Detailed User Guides & Workflows

Customer Workflow

1. Register/Log in as a **Customer**.
2. Click **"Place New Order"**. The form loads active pincodes dynamically.
3. Input package dimensions. Review the live cost breakdown showing base costs, per-kg rates, and COD surcharges.
4. Confirm placement. View the live order list.
5. Click any card to open the **Tracking Screen** showing real-time timeline steps and logs.
6. If an attempt fails, click **"Reschedule"** to pick a new date.

Agent Workflow

1. Log in as an **Agent**.
2. Toggle status between **AVAILABLE** and **OFFLINE** on your profile card.
3. View assigned shipments in the task panel.
4. Click **"Mark as Picked Up"**, **"In Transit"**, **"Out for Delivery"**, or **"Delivered/Failed"** to progress the package.

Admin Workflow

1. Log in as `admin@lastmile.com`.
2. **Manage Shipments**: View all orders, filter by status, manually re-assign agents, or override states.
3. **Zone Boundaries**: Create new service zones and assign pincodes.
4. **Rates & COD Config**: Adjust rate cards and COD surcharges dynamically.
5. **Delivery Agents**: Register new agent profiles and manage credentials.